

AIOps: An AI-Assisted DevOps Monitoring System with Anomaly Detection and LLM-Driven Diagnostics on AWS

Ketan Bhendarkar

Dept. of Computer Science and Engineering
MIT-ADT University, Pune, India
ketanbhendarkar2002@gmail.com

Piyush Singh

Dept. of Computer Science and Engineering
MIT-ADT University, Pune, India
singhpiyush19678@gmail.com

Ashwini Lawhale

Dept. of Computer Science and Engineering
MIT-ADT University, Pune, India
ashwinilawhale02@gmail.com

Prof. Manoj Shinde

Dept. of Computer Science and Engineering
MIT-ADT University, Pune, India
manoj.shinde@mituniversity.edu.in

Abstract—Our paper presents a DevOps monitoring platform that was designed and implemented using the help of an AI. The data acquisition stage is managed with the help of the Prometheus monitoring server along with the use of the Node Exporter. The backend comprises 18 endpoints that feed into the React 18 web application consisting of ten pages. The detection is done via the application of the Isolation Forest on 10,200 samples (10,000 normal + 200 anomalies) with five features. With regard to testing it on the test split the weighted F1 score was 0.9995 with 97.56% precision on the anomaly class and 100% recall. Per-inference latency does not exceed 25 ms. The system design aimed to go beyond mere detection and to provide some useful functionality. Therefore, our platform was enhanced with the ability to communicate via a large language model chatbot and eleven remote action triggers available right from the browser—from CPU stress test to a full load test of the entire system. Alert fatigue is avoided by giving individual cooldowns to each metric: a RAM-related alert will never be prevented by CPU alerts, or vice versa. The response time of all endpoints is kept below 35 ms. The idea of combining unsupervised anomaly detection, LLM-driven explanation, and remote actions proved to be practical for small clouds.

Index Terms—AIOps, DevOps, Anomaly Detection, Isolation Forest, Cloud Monitoring, Large Language Models, Real-Time Systems, Prometheus, AWS EC2

I. INTRODUCTION

Setting up cloud infrastructure has become trivial nowadays, but monitoring it is far from being so. One virtual machine can generate hundreds of metrics each several seconds. At such dynamic, there is no way an operator can cope with reading graphs manually, and raw data overwhelms the important messages [1].

As a workaround, static threshold-based rules have become a norm: define a metric value to trigger an alert when crossed. This solution works for some time until two of its issues show up; these are very familiar to everyone who has ever

spent nights on call. First of all, a rule based on one metric cannot take into account any context. It does not distinguish between a spike in CPU usage after a deploy and the same situation during steady traffic, while there is a big difference here. Secondly, once rules start firing too frequently, operators just ignore them, and MTTD becomes an issue [3].

A more intelligent approach to dealing with monitoring called AIOps can be defined as adding another layer to the existing infrastructure monitoring solutions [8]. However, our main observation related to most AIOps research is the absence of any actions taken after detecting an anomaly. However, this system has been designed to extend beyond simple detection. There are three extensions on top of basic detection:

- 1) A large language model chat assistant, which takes the current snapshot of the metric within the context of its system prompt and outputs a natural language interpretation of the detected situation.
- 2) Eleven remote action capabilities exposed through buttons in the dashboard, including stress tests, restarts, freeing memory, and flushing logs.
- 3) Email notification capabilities featuring a distinct cooldown mechanism for each metric, minimizing duplicate notifications without silencing notifications based on other metrics.

This is implemented on one AWS EC2 machine (Ubuntu 24.04). Storage is done by Prometheus v2.52, the backend is a Flask app for serving an API with the inference logic, and React 18 provides the dashboard interface. The model used to detect anomalies is a five-feature Isolation Forest, selected due to considerations laid out in Section III.

The following sections are arranged as follows. Section II

includes a literature review for threshold-based monitoring systems, ML-based anomaly detection, and current AIOps platforms. The system’s architecture and design of all components are described in Section III. Section IV features evaluation results. Finally, conclusions are given in Section V, followed by future work in Section VI.

II. LITERATURE REVIEW

A. Threshold-Based Monitoring

The first monitoring systems were based on manually written rules. As soon as some threshold was crossed, an alert would be triggered. They were inexpensive and straightforward, and that is the reason behind their longevity, but they required regular fine-tuning and failed to detect the interdependency of multiple indicators [5]. In cloud environments where autoscaling happens, and the workload is unpredictable, the approach proved to be very ineffective. False alarms increased, and compound errors (in which all monitored variables moved in tandem) remained undetected.

B. Anomaly Detection Using Machine Learning Algorithms

Both supervised and unsupervised methods are popular approaches in monitoring server telemetry. Supervised classification approaches such as random forest classifiers or support vector machines might seem appealing. However, as there is no source for a large amount of labeled data about system incidents, this route is hardly feasible [7]. The more realistic choice will thus be to go for unsupervised techniques. One such algorithm is the Isolation Forest suggested by Liu et al. [2]. The intuition behind it is quite simple—construct binary decision trees randomly, and use the property of outlying objects to exist in sparse regions that require less splitting. As a result, the algorithm is able to achieve sub-linear runtime in high dimensional feature spaces. It has been shown to effectively perform the task on telemetry data where the anomalies are much less numerous than regular data items [6].

C. Observability Tools for Cloud Infrastructure

With the evolution of cloud technology came advancements in its associated tooling. Prometheus established itself as the de facto metrics aggregation and query engine. Grafana became the standard dashboard solution, while distributed tracing platforms started providing insight into requests’ lifecycle [5]. Such tools offer high-quality performance, yet they fail to analyze the collected information. Human interpretation is still required, a process viable in simple settings but unsustainable in modern cloud infrastructure.

D. Modern AIOps Architectures

In 2017, Gartner coined the term AIOps to refer to the use of machine learning and automation techniques on top of monitoring data for IT operations [8]. To date, multiple prototypes, including industrial solutions and academic research, have successfully employed AIOps in event correlation in heterogeneous environments, anomaly detection in real time, and partial automation of runbook orchestration [9]. A few

key challenges remain persistent in the area. First, metrics, logs, and traces are considered separately rather than combined for comprehensive analysis. Second, ML model output is usually difficult for operators to understand because only numerical scores are provided. Finally, deployment itself poses difficulties, with small organizations often opting not to deploy such advanced tooling [10].

E. Gaps and Research Directions

Based on the review of the state-of-the-art literature in the field, we have identified four key gaps to address in our project. First, static models fail to adapt to the changing workload effectively. Second, there is no inherent natural language explanation capability for discovered anomalies. Third, dashboards typically lack any ability to execute an action directly on the system in response to an anomaly. Lastly, the current ecosystem consists of separate solutions, each responsible for a different part of the workflow. Our platform aims to resolve all four problems by integrating Prometheus-based metrics collection and Isolation Forest anomaly detection, providing an LLM-powered diagnostic experience, and enabling browser-accessible remote actions execution, all in a cohesive stack.

III. METHODOLOGY

A. System Architecture

The platform is divided into five layers: metric collection, backend processing, ML inference, alerting, and visualization. Fig. 1 shows the flow of data between them.

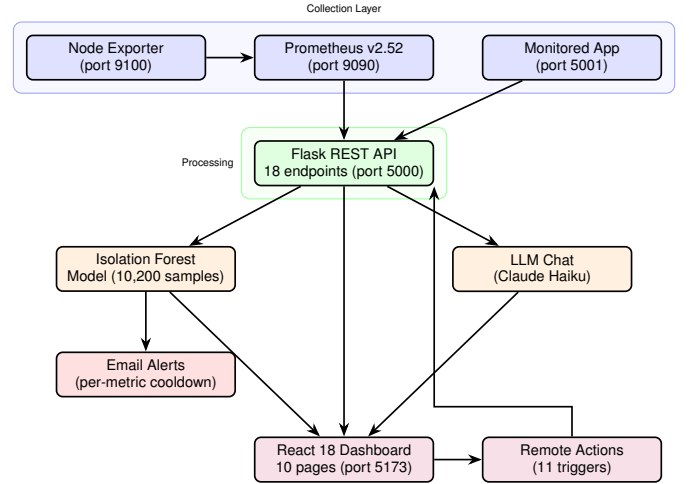


Fig. 1. System architecture showing the five-tier data flow from metric collection through ML inference, LLM diagnostics, alerting, and interactive visualization with closed-loop remote actions.

B. Infrastructure and Deployment

Everything runs on a single AWS EC2 t2.micro (1 vCPU, 1 GB RAM, Ubuntu 24.04 LTS, us-east-1). The instance was chosen deliberately—part of the goal was to show that this kind of monitoring does not need dedicated hardware to work. To survive reboots, four services are registered with systemd: node_exporter, prometheus,

aiops-backend (Flask, port 5000), and monitored-app (a small Flask application on port 5001 that acts as the thing being monitored). Prometheus is configured with a 15-second scrape interval pointing at Node Exporter on port 9100.

C. Metric Collection and PromQL Queries

Node Exporter exposes well over 500 kernel-level counters, most of which we do not need. The backend pulls a small curated subset using PromQL, the query language that Prometheus uses. Listing 1 contains the queries for the five features that get passed to the anomaly model.

```

1 # CPU utilization (%)
2 100 - (avg(rate(
3   node_cpu_seconds_total{mode="idle"}[5m]
4 )) * 100)
5
6 # Memory utilization (%)
7 (1 - node_memory_MemAvailable_bytes
8   / node_memory_MemTotal_bytes) * 100
9
10 # Disk utilization (%)
11 100 - (node_filesystem_avail_bytes{
12   mountpoint="/" /
13   node_filesystem_size_bytes{
14     mountpoint="/" } * 100)
15
16 # Network inbound rate (bytes/s)
17 rate(node_network_receive_bytes_total{
18   device="enx0"}[5m])
19
20 # System load average (1-min)
21 node_load1

```

Listing 1. PromQL queries for primary metrics.

On top of the five core metrics above, a separate extended endpoint exposes disk I/O throughput, read/write IOPS, network error and drop rates, TCP connection counts, and socket utilization. These are used by the Disk and Network pages in the dashboard.

D. Data Preprocessing

Before a metric vector reaches the model, it passes through a short preprocessing step. Prometheus occasionally returns nulls when a scrape is missed; in those cases the last valid reading is carried forward. Numerical features are then min-max normalized into $[0, 1]$, since unscaled values would give the larger-range features (like network bytes per second) a disproportionate influence on tree splits. Finally, the five features are assembled into a single row vector $\mathbf{x} = [\text{cpu}, \text{ram}, \text{disk}, \text{net_in}, \text{load}]$, aligned to a common 15-second timestamp.

E. Anomaly Detection with Isolation Forest

To implement the anomaly detector, the Isolation Forest algorithm [2] was used. The decision to adopt the algorithm was made because of lack of labeled production incident data, which would exclude any supervised solution. Additionally, Isolation Forest performs well in case of rare events detection, which corresponds to real life situation with server anomalies.

The training dataset consists of 10,000 normal instances and 200 anomalies. They correspond to random numbers from uniform distribution: normal instances' CPU usage falls into interval $[5, 30]\%$ and RAM usage to $[20, 55]\%$; the same

applies to anomalies but CPU usage is in $[70, 99]\%$ and RAM usage is in $[75, 98]\%$. The contamination is set to 0.02 as well as the rate of anomalies in the training dataset. The Isolation Forest has been trained with 100 trees and decision threshold set. Instance's anomaly score being above the threshold means its labeling as normal instance (+1); below the threshold the instance will be classified as anomalous (−1).

Confidence level is obtained through the equation below depending on raw anomaly score:

$$\text{confidence} = \min(100, \max(0, (1 - (s + 0.5)) \times 100)) \quad (1)$$

Model parameters are saved to the disk with `joblib` and loaded once the server is started up (size is around 1,010 KB). Since inference does not require constructing the forest, but rather querying it, the prediction time is kept minimal. Training process takes less than a second even on `t2.micro`.

F. LLM-Powered Diagnostic Chat

This chat assistant is one of the components that make the system different from a typical dashboard. The component itself is a page integrated into the dashboard that calls an LLM hosted on the cloud. When a user asks a question, the client first prepares an API call and appends a system prompt, which contains the current value of every metric in question—CPU usage, memory consumption, disk metrics, load levels, uptime duration, network speed measurements, anomaly flag, its confidence score, and current recommendation string shown by the dashboard.

For instance, in one of the CPU-stress testing runs, the chat assistant properly diagnosed the problem of `t2.micro` CPU credits running out, recommended certain diagnostic actions that confirmed the hypothesis, and indicated process-based diagnosis as the next logical step (see Fig. 4). Here, too, the LLM had no prior knowledge of the server under scrutiny.

G. Alert Generation and Email Notifications

The backend issues alerts based on thresholds that are either set by rules or fired based on ML-detected anomalies. Threshold values set up while designing the system are shown in Table I.

TABLE I
ALERT THRESHOLDS AND NOTIFICATION TRIGGERS

Metric	Warning	Critical	Email
CPU Usage	> 20%	> 35%	Yes
RAM Usage	> 58%	> 60%	Yes
Disk Usage	> 80%	> 90%	Yes
App Error Rate	—	> 5%	Yes
App Response	> 2000ms	—	Yes
ML Anomaly	—	Any	Yes

If a threshold gets exceeded, the backend fires off an e-mail notification that is a rich HTML message. It contains the severity badge, the name and current value of the metric, UTC timestamp, a recommendation of what actions should be taken next as provided by the AI, and server-related metadata including hostname, region, and instance type. In

early versions of this system, a single cooldown timer was being used for all alerts; therefore, if one alert had already been issued for a noisy CPU usage, other alerts such as RAM would be suppressed. Now each metric maintains its own cooldown timer of 300 seconds in a Python dictionary. A sample email alert sent in response to one of the memory stress tests is illustrated in Fig. 5.

H. Remote-Action Layer

Eleven remote action endpoints are exposed by the backend (Table II) and allow users to issue commands from the dashboard interface to carry out maintenance or trigger faults, without logging into the instance via SSH. These actions are mapped one-to-one with a button click on the dashboard’s Actions page, and the results are displayed in a live console-log widget underneath the buttons.

TABLE II
REMOTE ACTION TRIGGERS

Action	Effect
CPU Spike	<code>stress --cpu 4 --timeout 300s</code>
Memory Spike	8× allocation calls (~800MB)
Free Memory	Releases all allocated blocks
App Errors	20× HTTP 500 injections
Slow Response	3× 3-second delay injections
Restart App	Kills and restarts monitored app
Clear Logs	<code>journalctl --vacuum-size=50M</code>
CPU Load	10× prime-computation requests
App Status	Returns current app health JSON
Full Stress	Concurrent CPU + RAM + Disk + App load
Stop All	Terminates all stress processes

This closed loop design is for two different use cases. It is suitable for live demos where the presenter wants to show end to end behavior in a limited time window, but it is also suitable for real chaos engineering workflows where an operator wants to inject a fault, observe the detection response, ask the LLM about what happened, and remediate. All this happens in the same browser tab.

I. Visualization Dashboard

The frontend is a React 18 single page application, built with Vite, and communicates with the backend over Axios. Charts are plotted using Recharts library with gradient filled area plots. The dashboard contains ten pages in total: Overview (8 metric cards + sparklines and 4 time series charts), CPU, Memory, Network (throughput, errors, drops, TCP connections), Disk (space, I/O rates, IOPS), Processes (a top ten table sorted by CPU share), Alerts, AI Anomaly (status, confidence meter, input-feature grid, recommendation), AI Chat, and Actions.

There are two polling intervals available. The lightweight endpoints are hit every 15 seconds which is the Prometheus scrape interval. The lightweight endpoints are `/api/metrics/live`, `/api/processes`, `/api/alerts`, `/api/anomaly`. The history endpoint is heavier, so it is polled every minute. There is also an export

button on the toolbar that will save a PNG of the current view or a formatted PDF health report.

A dark-themed, visually heavy design is used, with a near-black background (#0a0a12), violet as the primary accent (#8b5cf6), glassmorphism cards with backdrop blur, and the Outfit display typeface with JetBrains Mono for numerical values.

IV. RESULTS

The evaluation was performed along three dimensions: accuracy of the anomaly detector, latency of the REST API and end-to-end behavior in a stress-test scenario.

A. Anomaly Detection Performance

The 10,200-sample synthetic dataset was split 80/20 with stratification for the detector and trained on the 80% training set of the dataset with the Isolation Forest. In Table III we report results on the held-out test set.

TABLE III
ISOLATION FOREST CLASSIFICATION METRICS (TEST SET, $n = 2,040$)

Class	Precision	Recall	F1-Score	Support
Normal	1.0000	0.9995	0.9997	2,000
Anomaly	0.9756	1.0000	0.9877	40
Weighted Avg	0.9995	0.9995	0.9995	2,040

It is further decomposed in the confusion matrix (Table IV). There is exactly one false positive, i.e., a normal sample misclassified as anomalous, and no false negatives. This is in line with the design priority for the system. In a monitoring context, missing a real anomaly is far worse than raising a false alarm occasionally. The model errs on the right side.

TABLE IV
CONFUSION MATRIX

	Pred. Normal	Pred. Anomaly
True Normal	1,999	1
True Anomaly	0	40

The overall accuracy is 99.95%. The serialized model file is about 1,010 KB and training on the `t2.micro` takes about 0.92 seconds. That combo makes retraining cheap enough to cycle through even on this constrained hardware, which matters later if production data eventually gets mixed in.

B. API Delays

To get a feel for the responsiveness of the API under average load, each REST endpoint was hit 20 times in a row and the round trip time averaged. The numbers are shown in Table V.

All endpoints are under 35 ms which is well within the 15 second polling window the dashboard uses. The endpoint that loads the serialized model in practice and runs inference on the current metric vector is `/api/anomaly` and its average is 24.30 ms—confirmed what we wanted to check early: Isolation Forest inference adds very little to the request

TABLE V
MEAN API RESPONSE LATENCY (20 TRIALS PER ENDPOINT)

Endpoint	Mean Latency (ms)
/api/health	1.20
/api/metrics/live	21.70
/api/metrics/history	17.90
/api/anomaly	24.30
/api/processes	8.60
/api/alerts	19.70
/api/metrics/extended	34.70

path. The slowest endpoint is `/api/metrics/extended` with 34.70 ms. This is expected behavior, as that endpoint diverges into multiple PromQL sub-queries in parallel (disk I/O, network errors, TCP state counters) each with its own round trip to Prometheus.

C. Stress-Test Scenario

The “Full System Stress” button was pressed on the Actions page for the end-to-end stress test. A single button triggers a simultaneous CPU stress (`stress --cpu 4`), memory allocation (~800 MB), disk-write load with `dd` and application-level error injection. In one polling cycle (15 seconds) the dashboard collected the following:

- CPU utilization increased from ~22% baseline to 67.2%.
- RAM usage went up from ~56% to 61.0%.
- System load average increased to 10.4 from 0.03.
- The Isolation Forest model classified the state as anomalous with 68.8% confidence (Fig. 3).
- The AI recommendation panel said: “High CPU detected—check for runaway processes. Consider restarting nginx or app server.”
- Two independent email alerts (CPU critical, RAM critical) were dispatched within two minutes, each carrying the metric value, timestamp, severity badge, and AI recommendation (Fig. 5).

Pressing “Stop All Stress” cancels all injected loads. Metrics returned to baseline within 30 seconds and the anomaly flag cleared itself on the next polling cycle. The entire loop (fault injection, detection, explanation, remediation) was under 3 minutes and required no SSH access to the instance.

D. Dashboard and Operational Views

Fig. 2 shows the Overview page during normal operation. Eight metric cards are at the top, displaying sparkline trends. Live time-series charts fill the lower half. In Fig. 4, the AI Chat assistant answers a question about CPU use during a period of mild load. The diagnostic commands it suggests come directly from the injected metric context. Fig. 6 displays the Actions page, featuring eleven remote triggers and a live console log that records the output of recent operations.

V. CONCLUSION

The work presented here is an AIOps monitoring platform built around five operational stages: metric collection, anomaly

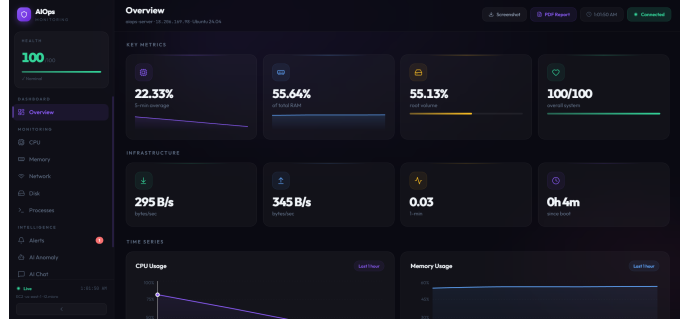


Fig. 2. Dashboard Overview page displaying key metric cards, infrastructure indicators, and time-series charts during normal operation.

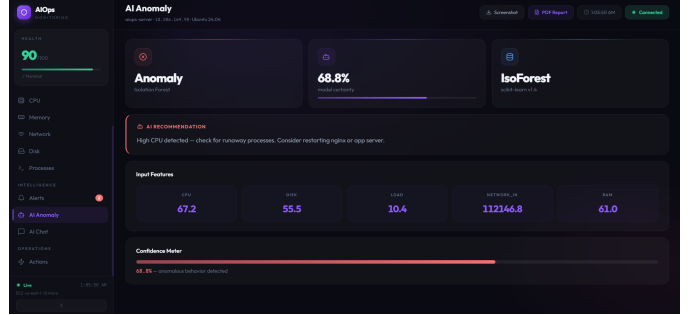


Fig. 3. AI Anomaly page during a full-system stress test: Isolation Forest detects anomalous behavior at 68.8% confidence with input feature values and an AI-generated recommendation.

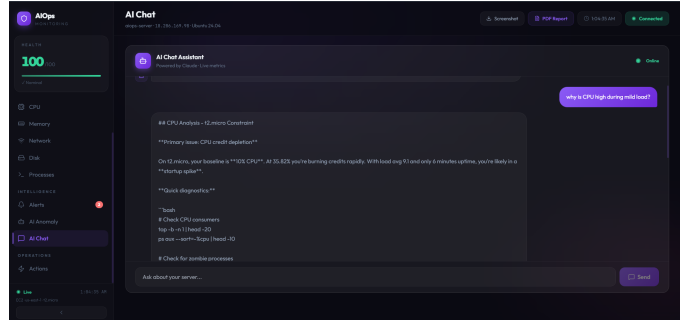


Fig. 4. AI Chat assistant analyzing CPU utilization during mild load, identifying t2.micro credit depletion and suggesting diagnostic commands based on live metric context.

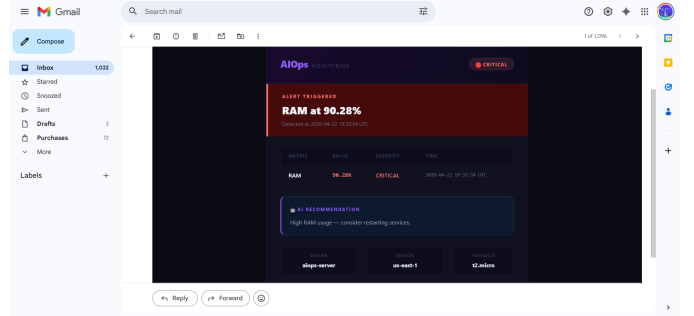


Fig. 5. Rich HTML email alert received during a memory stress test, showing severity badge, metric details, AI recommendation, and server metadata.

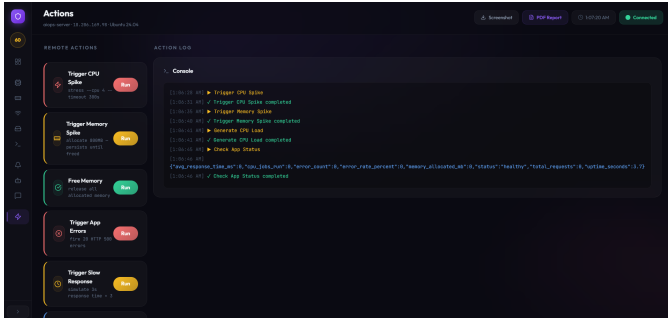


Fig. 6. Remote Actions page with eleven server-management triggers and a live console log displaying recent action output.

detection, contextual diagnosis, alerting, and closed-loop remediation. The Isolation Forest component performed well on the evaluation data, producing a weighted F1 of 0.9995 with zero false negatives. It adds roughly 24 ms of inference latency per request, which is small enough to go unnoticed during normal polling.

By pairing that detector with an LLM chat assistant and sending live metric context into the prompt, the platform offers something that plain dashboards and rule-based alerts cannot provide: an interpretable explanation of what the numbers mean at that moment. The remote-action layer then completes the process by allowing operators to take action from the same browser session.

One of the design goals was to show that effective AI-augmented monitoring does not require a large cluster or a dedicated observability cloud bill to work. Running the entire stack on a single `t2.micro` demonstrates that quite well. The modular separation between collection, processing, inference, and presentation should also make it easy to move individual components onto larger instances, into containers, or across multiple nodes when necessary.

VI. FUTURE WORK

There are several directions in which the current platform can be extended.

Root-cause analysis. At present the system flags an anomaly without pointing to what caused it. Layering on causal inference or attention-based models would allow the anomaly to be traced back to a specific process, configuration change, or deployment event, which would save meaningful time during investigation.

Log-based detection. Numerical metrics only tell part of the story. Applying NLP techniques to system and application logs would catch semantic anomalies too, for instance novel error messages or unusual log patterns that numerical features on their own cannot capture.

Predictive forecasting. Time-series models such as LSTMs or Facebook’s Prophet could be used to forecast where metrics are heading in the next few minutes. If the forecast crosses a threshold, operators could be notified in advance and scale or patch before an actual incident occurs.

Self-healing automation. The remote-action layer was built with a human in the loop, but the architecture would support autonomous operation too. For well-understood anomalies—sustained memory pressure on a web worker, for example—the remediation step could be triggered automatically once the detector’s confidence is high enough. Doing this carefully would reduce mean time to resolution (MTTR) without requiring an operator.

Multi-cloud and container support. Currently the collection layer only looks at one EC2 instance. Extending it to scrape Kubernetes pods, and adding adapters for other cloud providers, would make the same pipeline useful for real production workloads.

Continuous model adaptation. Finally, the anomaly model is trained once on synthetic data and then left alone. Introducing online learning, or at least periodic retraining on recent production traffic, would let the model keep pace with workload drift instead of relying on a static training distribution.

ACKNOWLEDGMENT

The authors are grateful to Prof. Manoj Shinde for his guidance and mentorship throughout the development of this project.

REFERENCES

- [1] M. Chen, S. Mao, and Y. Liu, “Big data: A survey,” *Mobile Networks and Applications*, vol. 19, no. 2, pp. 171–209, 2014.
- [2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.
- [3] G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. Portland, OR: IT Revolution Press, 2016.
- [4] B. Burns, J. Beda, and K. Hightower, *Kubernetes: Up and Running*, 2nd ed. Sebastopol, CA: O’Reilly Media, 2019.
- [5] Prometheus Authors, “Prometheus documentation,” 2023. [Online]. Available: <https://prometheus.io/docs/>
- [6] Scikit-learn Developers, “Isolation forest,” *scikit-learn documentation*, 2024. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- [7] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] Y. Dang, Q. Lin, and P. Huang, “AIOps: Real-world challenges and research innovations,” in *Proc. 41st Int. Conf. Software Engineering: Companion (ICSE-Companion)*, Montreal, Canada, 2019, pp. 4–5.
- [9] P. Notaro, J. Cardoso, and M. Gerndt, “A systematic mapping study in AIOps,” in *Proc. Int. Conf. Service-Oriented Computing (ICSOC)*, 2021, pp. 110–123.
- [10] Y. Lyu *et al.*, “Towards a unified framework for AIOps: Challenges and opportunities,” *arXiv preprint arXiv:2112.00733*, 2021.
- [11] Pallets Projects, “Flask documentation,” 2024. [Online]. Available: <https://flask.palletsprojects.com/>
- [12] Meta Platforms, “React documentation,” 2024. [Online]. Available: <https://react.dev/>
- [13] Prometheus Authors, “Node exporter,” 2023. [Online]. Available: https://github.com/prometheus/node_exporter
- [14] Amazon Web Services, “Amazon EC2 documentation,” 2024. [Online]. Available: <https://docs.aws.amazon.com/ec2/>